

Manual Tests

TP3 Deliverable 5.4.3

CSE 360 FoundationsS26 Team Project Phase 3

Purpose. This document lists the manual tests that are still required after the current JUnit and semi-automated tests. The automated tests already cover entity behavior, validator behavior, persistence behavior, and selected authorization logic. These manual tests focus on JavaFX scene transitions, modal dialogs, TableView interaction, invitation-based onboarding, and end-to-end staff workflows that are difficult to validate cleanly with the current JUnit infrastructure.

Automated coverage already present. The current test set covers Post and Reply entity behavior, validator classes, core database behavior, discussion-board persistence, and selected controller authorization rules. Manual tests below are intentionally limited to behaviors that depend on visible GUI state, human review of alerts/dialogs, or multi-window application flow.

1. Manual Testing Strategy

1.1 Environment. Run the TP3 application from Eclipse using the same JavaFX and H2 configuration used for normal execution. Unless a test says otherwise, begin from a working build with the database reachable and the JavaFX UI launching successfully.

1.2 Test data. Use realistic names, email addresses, usernames, post headers, post bodies, and replies. For staff-facing tests, use at least one Admin account and at least two non-admin user accounts so that permission and review workflows can be observed.

1.3 Success rule. A manual test succeeds only when the visible behavior, alerts, table content, and page transition all match the expected result described in the test outline.

2. Manual Test MT-01: First Admin Account Setup

Why manual test is needed. This flow begins at application startup and depends on JavaFX page creation, two password fields, live validation messaging, and the transition from the first-admin page into the normal application flow. The current JUnit tests do not verify this visible startup behavior.

What the test does. Confirms that a brand-new system can create the initial administrator account through the First Admin page and move forward without hard-coded credentials.

How to run the test.

- Start with an empty or reset database so no users exist.
- Launch the application.
- On the First Admin page, enter a valid username and matching valid passwords.
- Select the setup action for the administrator account.

Input to provide.

- Username: adminLead
- Password 1: StrongPass1
- Password 2: StrongPass1

Expected output.

- No invalid-input alert remains on screen.
- The administrator account is created.
- The application transitions out of the startup page into the next expected account-update or admin flow.

Pass/fail determination. Pass if the account is created and the UI advances normally. Fail if the page stays blocked, passwords are mishandled, or the user is not routed forward.

3. Manual Test MT-02: Admin Sends Invitation Code

Why manual test is needed. This behavior depends on the Admin Home page, user input, role selection, alert dialogs, and dynamic status labels. Those GUI details are not reliably validated by the automated tests.

What the test does. Verifies that an administrator can send an invitation to a new user and that the UI reports the generated code and updated invitation count.

How to run the test.

- Log in as an administrator.
- Open Admin Home.
- Enter a valid email address.
- Select a role from the invitation role combo box.
- Press Send Invitation.

Input to provide.

- Email: new.role1.user@example.com
- Role: Role1

Expected output.

- A success/information alert appears showing an invitation code and the target role.
- The outstanding-invitations count increases or otherwise reflects the new invitation.
- The email input field is cleared after the invitation is processed.

Pass/fail determination. Pass if the alert appears with a generated code and the admin page state updates correctly. Fail if no code is shown, the wrong role is attached, or the status count does not update.

4. Manual Test MT-03: Invitation-Based New Account Creation

Why manual test is needed. This is a multi-screen workflow involving the login page, invitation-code lookup, the New Account page, live username/password validation, and final account creation. It is broader than the existing unit tests.

What the test does. Checks that a user with a valid invitation code can create an account for the invited role and continue into the normal application flow.

How to run the test.

- From the login/start page, enter a valid invitation code created in MT-02.
- Proceed to the New Account page.
- Enter a valid username and matching valid passwords.
- Complete account setup.

Input to provide.

- Invitation code: use the code produced in MT-02
- Username: invitedRole1
- Password 1: StrongPass1
- Password 2: StrongPass1

Expected output.

- The New Account page accepts the invitation code and identifies the invited role.
- The account is created successfully.
- The application routes the user into the next expected account-update or role-home flow.

Pass/fail determination. Pass if the invited user account is created with the expected role and the UI proceeds normally. Fail if the invitation is rejected incorrectly, validation blocks correct input, or the created account lacks the invited role.

5. Manual Test MT-04: Multiple Role Dispatch

Why manual test is needed. The multiple-role dispatch page depends on dynamic combo-box population and scene routing, which are not covered by the current automated tests.

What the test does. Verifies that a user who has multiple roles is shown the Multiple Role Dispatch page and can enter the selected role's home page.

How to run the test.

- Use an account that has at least two roles assigned.
- Log in with that account.
- When the dispatch page appears, inspect the role-selection list.
- Choose one role and perform it.
- Return or log out, then repeat for the other role.

Input to provide.

- Account with Admin + Role1 or Role1 + Role2

Expected output.

- Only the roles actually assigned to the user appear in the combo box.
- Selecting a role opens the matching role home page.
- The user is not sent to a role that was not selected.

Pass/fail determination. Pass if the available roles are correct and each selection opens the correct page. Fail if the list is wrong or the dispatch page routes to the wrong home page.

6. Manual Test MT-05: User Account Update Dialogs and Validation

Why manual test is needed. Account-update behavior depends on multiple dialog boxes, JavaFX alerts, field-specific validators, and visible label refreshes on the user page. These are not fully covered by unit tests.

What the test does. Confirms that the user can update personal details and that invalid values are blocked with appropriate feedback.

How to run the test.

- Log in as any user and open Account Update.
- Attempt one valid update for a name field and one valid email update.
- Attempt at least one invalid name or email update.
- Attempt a password change with mismatched values, then with matching valid values.

Input to provide.

- Valid first name: Alex
- Valid email: alex.user@example.com
- Invalid email example: alex@@example
- Mismatched password example: StrongPass1 / StrongPass2

Expected output.

- Valid inputs update the visible labels and/or stored values.
- Invalid inputs trigger an error alert or helper message and do not overwrite the current value.
- A successful password change produces the expected confirmation.

Pass/fail determination. Pass if valid edits persist and invalid edits are rejected cleanly. Fail if labels do not refresh, invalid data is accepted, or password feedback is inconsistent.

7. Manual Test MT-06: Admin User Management - Activate and Deactivate

Why manual test is needed. The Admin Users dialog uses a TableView, selection state, buttons, filtering, and visible active-status changes. This is awkward to validate with the current JUnit setup and is better verified directly in the UI.

What the test does. Verifies that an administrator can deactivate an active user and reactivate the same user using the admin-management dialog.

How to run the test.

- Log in as an administrator.
- Open the Admin Users dialog.
- Select a non-admin user from the table.
- Deactivate the user and refresh the view if needed.
- Reactivate the same user and refresh again.

Input to provide.

- Target user: any existing non-admin account

Expected output.

- The dialog reports the success of each action.
- The selected user's active state changes accordingly in the table or after refresh.
- No unrelated user is modified.

Pass/fail determination. Pass if the chosen user toggles between active and inactive correctly. Fail if the wrong user changes, no visible status change occurs, or the dialog reports success without changing state.

8. Manual Test MT-07: Admin User Management - Change Role and Reset Password

Why manual test is needed. This workflow includes role-selection dialogs, temporary password generation, visible confirmation dialogs, and follow-on account behavior. It is a staff-facing workflow that should be demonstrated end-to-end.

What the test does. Checks that an administrator can change a user's role and issue a one-time or temporary password reset through the admin-management tools.

How to run the test.

- Log in as an administrator.
- Open the Admin Users dialog.
- Select a target user.
- Use the change-role action to grant or replace a role.
- Use the reset-password action and record the generated value.
- Optionally, log in as the target user with the reset value if the flow supports it.

Input to provide.

- Target user: existing non-admin user
- New role: Role1 or Role2

Expected output.

- The role change is acknowledged and is visible in the table after refresh.
- The reset-password action produces a new temporary password/token.
- The target account reflects the password-reset action as designed.

Pass/fail determination. Pass if the role change and password reset both complete for the selected user only. Fail if the wrong role is applied, no reset value is shown, or the workflow silently fails.

9. Manual Test MT-08: Create, Search, and Filter Discussion Posts

Why manual test is needed. The post list view depends on JavaFX TableView rendering, search field behavior, solved-status filtering, and modal create-post dialogs. These visible UI behaviors are not fully covered by the database/entity tests.

What the test does. Verifies that a user can create a discussion post and then locate it using the search and solved-status filter controls.

How to run the test.

- Log in as a user who can access the discussion board.
- Open the discussion board.
- Create a post with a distinctive header and body.
- Use the search field to filter by part of the header.
- Toggle the solved-status filter between All, Solved, and Unsolved.

Input to provide.

- Header: TP3 Manual Test Unique Header
- Body: This post is being used to validate search and filter behavior in the TP3 discussion board.

Expected output.

- The new post appears in the post list.

- The search field reduces the list to matching items.
- The solved filter shows or hides the post according to its current solved state.

Pass/fail determination. Pass if the post is visible, searchable, and filtered correctly. Fail if it does not appear, search does nothing, or the solved filter behaves incorrectly.

10. Manual Test MT-09: Edit/Delete Post and Reply Through the UI

Why manual test is needed. The current automated tests cover entity and database mutation, but not the actual JavaFX dialogs, button bindings, table refreshes, and user-facing delete/edit flow.

What the test does. Checks that a user can edit and delete their own posts and replies through the visible discussion-board controls.

How to run the test.

- Create a post as the current user.
- Open that post and create a reply under it.
- Use the edit controls to change the post and then the reply.
- Use the delete controls to remove the reply and then the post.

Input to provide.

- Original post header/body and revised post header/body
- Original reply body and revised reply body

Expected output.

- Edits are visible immediately in the open post view or post list.
- Deleted content disappears from the table/list after the action.
- No error alert appears for the owner performing the action.

Pass/fail determination. Pass if owned content can be edited and deleted and the UI refreshes correctly. Fail if changes do not persist visually or if the content remains after deletion.

11. Manual Test MT-10: Non-Owner Cannot Edit or Delete Another User's Content

Why manual test is needed. Although there are controller authorization tests, the visible staff/student experience still needs manual confirmation because the real behavior involves button availability, alerts, and the active logged-in session.

What the test does. Verifies that a non-owner, non-admin user is blocked from editing or deleting another user's post or reply and receives an appropriate warning.

How to run the test.

- Log in as User A and create a post and reply content if needed.
- Log out and log in as User B, who is not an admin.
- Open User A's content from the discussion board.
- Attempt edit and delete actions on content owned by User A.

Input to provide.

- Two different non-admin accounts

Expected output.

- The non-owner is prevented from completing the protected action.
- A warning or error alert explains that only the original poster or an admin may perform the action.

Pass/fail determination. Pass if the non-owner cannot complete the edit/delete and the content remains unchanged. Fail if the non-owner successfully changes or removes content.

12. Manual Test MT-11: Mark Discussion Resolved and Reopen It

Why manual test is needed. Resolved/reopen behavior affects visible buttons, labels, post-list text, and post-view actions. These are UI state transitions that need direct observation.

What the test does. Checks that the owner or admin can mark a post resolved, observe the solved indicator, and then reopen it.

How to run the test.

- Open a post as its owner or as an administrator.
- Use the Mark Resolved action.
- Return to the post list and confirm solved status is visible.
- Reopen the same discussion and verify the solved indicator disappears.

Input to provide.

- Existing post owned by the current user or accessible to an admin

Expected output.

- The UI changes from unresolved to solved and back again.
- The post header/list text reflects the solved marker when applicable.
- Available buttons update appropriately for the post state.

Pass/fail determination. Pass if solved/reopen transitions are visible and consistent in both the post view and the list. Fail if the state changes are not reflected in the interface.

13. Manual Test MT-12: Report Abuse and Admin Review of Reports

Why manual test is needed. This is a staff-facing end-to-end workflow that depends on multiple windows, shared content identifiers, report accumulation, and the report-review UI. It is not fully covered by the current automated tests.

What the test does. Verifies that a user can report a post or reply and that an administrator can later open the reports view and inspect the flagged content.

How to run the test.

- As a non-admin user, open a post or reply that can be reported.
- Trigger the Report Abuse action.
- Log out and log in as an administrator.
- Open Admin Home and then open the reports/review dialog.
- Inspect the reported entry.

Input to provide.

- Existing reportable post or reply

Expected output.

- The content is recorded as reported.
- The administrator can open the reports dialog and see the reported item.
- The report count and summary information correspond to the content that was reported.

Pass/fail determination. Pass if the report appears in the admin review screen and points to the correct content. Fail if the report is lost, attached to the wrong item, or not visible to staff.